NATIONAL UNIVERSITY OF SINGAPORE

CS3230: Design and Analysis of Algorithms

(Semester 1: AY2015/16)

Time Allowed: 120 Minutes

INSTRUCTIONS TO CANDIDATES

a) This paper consists of **FOUR (4)** questions and **SIX (6)** printed pages including this page. Answer all questions.

b) You have 120 minutes to earn 80 marks. Do not spend too much time on any problem. Read them all through first and attack them in the order that allows you to make the most progress.

c) Show your work, as partial credit will be given. You will be graded on the correctness and efficiency of your answers, and also on your clarity. Be clear and always explain your reasoning.

d) This is a **CLOSED BOOK** exam but you are allowed to bring in **TWO** handwritten (on both sides) A4 crib sheet.

e) You may use an approved calculator.

Problem 1. True or False, and Justify (32 Marks)

Circle **T** (true) or **F** (false) for each of the following statements and justify your answer. Your justification is worth more than your true or false answer. You will not receive any mark if the justification is poor or missing, even if you answer true or false correctly.

a) **T F**

Consider the following variant of merge-sort. Instead of recursively sorting two halves of the array and merging them, we divide the array into three equal parts and recursively sort them. Let the outcome of recursively sorting the three parts be *A*, *B*, and *C* respectively. We merge *A* and *B* to get *D*, then merge *D* and *C* to obtain *E*, and return *E* as the outcome. The runtime of this algorithm is still $\Theta(n \lg n)$.

Solution: True.

The algorithm recursively solves three subproblems of size n/3 and takes $\Theta(n)$ to first merge *A* and *B*, and a further $\Theta(n)$ to merge *D* and *C*. Hence the recurrence is $T(n) = 3T\left(\frac{n}{3}\right) + \Theta(n)$. By Master theorem case 2, the solution of the recurrence is $T(n) = \Theta(n \lg n)$.

b) **T F**

$T(n) = 4T\left(\frac{n}{2}\right) + \Theta(n \ln n)$ implies $T(n) = \Theta(n \log^2 n)$.

Solution: False.

$f(n) = n \lg n = O(n^{\log_2 4 - \epsilon}) = O(n^{2-\epsilon})$. Hence case 1 of the Master theorem applies and $T(n) = \Theta(n^2)$.

c) **T F**

$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + \Theta(n)$ implies $T(n) = O(n)$.

Solution: True.

We show that $T(n) = O(n)$ by substitution. Assume as inductive hypothesis that $T(k) \leq Ck$ for all $k < n$. Then

$$T(n) \leq T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + cn$$
$$\leq \frac{Cn}{2} + \frac{Cn}{3} + cn$$
$$= \frac{5Cn}{6} + cn$$
$$= Cn - \left(\frac{Cn}{6} - cn\right).$$

It is possible to set C large enough for both the base cases to hold and for the term in the bracket is positive. Hence $T(n) \leq Cn = O(n)$.

Comment: Note that the two constants C and c are different.
Some students use a recursion tree to justify – okay if properly done.

d) **T F**

It is possible to output the k largest elements in an array in linear time.

Solution: True.

Use worst case linear time select to find the k-th largest element in $\Theta(n)$. Then use partition to find all the elements larger than the k-th largest element in $\Theta(n)$. The total runtime is $\Theta(n)$.

e) **T F**

Assume that you would like to sort n integers. The range of the integers to be sorted is in $[1,n^4]$. It is possible to sort the numbers in $O(n)$ time.

Solution: True.

Use radix sort. Radix sort runs $O\left(\frac{\log M}{\log n}\right)$ iterations of counting sort where M is the range of the elements. Hence it runs O(1) iterations of counting sort where each iteration of counting sort takes $O(n)$ time.

Comment: Should justify why the range $[1,n^4]$ still allows linear time sorting with radix sort.

f) **T F**

The *amortized cost* of the insert operation in dynamic tables is O(1). Therefore it is appropriate to use for real-time applications where each insert operations need to be done in O(1) *worst case* time.

Solution: False.

Amortize cost of O(1) does not imply worst case cost of O(1). In particular, for dynamic tables, the worst case cost of insert is O(n).

g) **T F**

The worst case runtime for performing *m* SEARCH operations for a self-organizing list with *n* keys using move-to-front is asymptotically as good as the worst case runtime for performing *m* SEARCH operations for a balanced search tree with *n* keys because move-to-front is 4-competitive.

Solution: False.

4-competitive only refers to being competitive with the optimal self-organizing list. In the worst case, the runtime of the optimal self-organizing list algorithm is $\Theta(mn)$ while a balanced tree takes $\Theta(m \log n)$ time.

h) **T F**

Consider the following algorithm for finding the longest increasing subsequence. We first initialize an array $L[1..n]$ to -1 and $L[1] = 1$. We will use the array L to store the length of the longest increasing subsequence.

```
LIS(A,i)
if L(i) ≠ -1
   return L[i]
max = 1
for j = i-1 downto 1
   x = LIS(A,j)
   if A[i] > A[j] and x + 1 > max
      max = x + 1
L[i] = max
return L[i]
```

The runtime for LIS(A,n) is $O(n)$ as there are only n subproblems to be solved.

Solution: False.

There are *n* subproblems, but the amount of work done in each subproblem is $O(n)$ and hence the runtime is $O(n^2)$. In fact, the runtime is $\theta(n^2)$. Each of the n subproblems need to be solved and subproblem *i* requires a loop of *i*-1 iterations. Hence total runtime is $\Omega(\sum_{i=1}^{n}(i-1)) = \Omega(n^2)$.

Problem 2. Algorithms in Context (16 Marks)

The design of efficient algorithms often requires choices. What is efficient in one context may be suboptimal in another. For each of the following, give one context where one algorithm may be preferable to another algorithm together with a reason to support the preference. You should describe contexts that as general as possible. Contexts that are overly restrictive may not be awarded full marks.

Example: When might insertion sort be preferable to merge sort?
Insertion sort might be preferable to merge sort when space is a limiting resource because unlike merge sort, insertion sort operates in-place.

a) When might merge sort be preferable to radix sort?

Solution: Merge sort might be preferable to radix sort when the range of the input elements is very large because radix sort has a runtime of $\Theta\left(\frac{\log M}{\log n} n\right)$ where M is the range and n is the number of elements to be sorted, while merge sort has a runtime of $O(n \log n)$ regardless of the range of the input elements.

b) When might counting sort be preferable to merge sort?

Solution: Counting sort might be preferable to merge sort when the range of input elements is very small because counting sort has a runtime of $\Theta(n + k)$ where k is the range while merge sort has a runtime of $O(n \log n)$.

c) When might randomized select be preferable to worst case linear time select?

Solution: Randomized select might be preferable to worst case linear time select when the performance criterion we are interested in is the expected runtime because randomized select also has a linear expected runtime but with a much smaller constant factor than worst case linear time select.

d) When might memoization be preferable to dynamic programming for solving the same problem?

Solution: Memoization might be preferable to dynamic programming for solving a problem when the number of subproblems that actually need to be solved to form the final solution is small in comparison to all possible subproblems, as memoization only solves subproblems that are required to form the final solution while dynamic programming solves all subproblems.

Problem 3. Canteen Placement (24 Marks)

SET-COVER: An instance (X,F) of a set cover problem consists of a finite set X and a family F of subsets of X, such that every element of X belongs to at least one subset in F, i.e. $X = \bigcup_{S \in F} S$. A subset C covers X if every element of X belongs to at least one subset of C, i.e. $X = \bigcup_{S \in C} S$. The set cover problem asks if there is a subset of size k that covers X.

VERTEX-COVER: A vertex cover of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that for every $(u, v) \in E$, either $u \in V'$ or $v \in V'$ (or both), i.e. every edge is covered. The vertex cover problem asks if there is a subset V' of size k that covers every edge in E.

VERTEX-COVER is known to be NP-complete.

CANTEEN-PLACEMENT: An instance of a canteen placement problem consists of a finite set $S = \{(x_1,y_1), \ldots, (x_n,y_n)\}$ of coordinates corresponding to building locations in an area. The CANTEEN-PLACEMENT problem asks if there is a subset $S' \subseteq S$ of size k such that every element in S is within a Euclidean distance of d from an element in S'. This correspond to placing k canteens in k buildings such that there is no building that is located further than a distance d from a canteen.

a) Argue that SET-COVER belongs to NP. (4 marks)

   Solution: Assume that a cover C of size k exists. Then we can use the list of subsets in C as a certificate. To verify, we need to check that each element of X is in one of the subsets in C. This takes polynomial time.

b) Show that SET-COVER is NP-hard, using VERTEX-COVER in the reduction. (12 marks)

   Solution: We reduce VERTEX-COVER to SET-COVER. First construct a set X consisting of the edges in G. For each vertex u, construct a subset of elements in X from all edges (u,v) that are incident to u. We have $|V|$ such subsets. Solve the SET-COVER problem for the same value of k. This transformation takes time polynomial in the size of the VERTEX-COVER problem.

   We now argue that a cover of size k for the VERTEX-COVER instance α implies a cover of size k for the corresponding SET-COVER instance β. Consider the k subsets associated with the vertices in α. As each subset consist of edges incident to one of the vertices in the vertex cover, the union of the subsets consist of all the edges in the graph G in α and hence all the elements of X in β.

   Similarly a cover of size k for the SET-COVER instance β implies cover of size k for the corresponding VERTEX-COVER instance α. To see this, note the each subset in the cover C of β consist of exactly the edges covered by one vertex in α. The union of the k subsets covers all elements in X, hence the union of the edges incident to the k corresponding vertices in α covers all the edges in the graph.

c) Show how the CANTEEN-PLACEMENT problem can be formulated as a SET-COVER problem. You may simply describe what the sets in SET-COVER correspond to, without proving that the problems are equivalent. (4 marks)

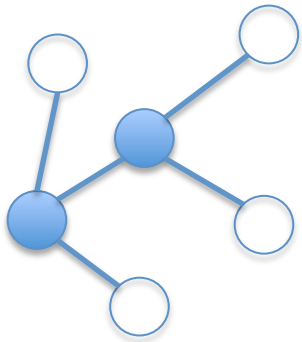
Solution: For each location (x_i,y_i) in the CANTEEN-PLACEMENT problem, construct a set for the SET-COVER problem consisting of all locations within a distance d from (x_i,y_i) (including (x_i,y_i) itself). The collection of n such subsets form F in the SET-COVER problem while the set S in the CANTEEN-PLACEMENT problem forms the set X in the SET-COVER problem.

d) CANTEEN-PLACEMENT can be reduced to SET-COVER in polynomial time. Given that SET-COVER is NP-complete, does that imply that the CANTEEN-PLACEMENT is NP-hard? (4 marks)

Solution: No, the reduction goes the wrong way. To show that CANTEEN-PLACEMENT is NP-hard, we need to reduce SET-COVER to CANTEEN-PLACEMENT in polynomial time.

Problem 4. Minimum Vertex Cover for Trees (8 Marks)

Given a tree $T = (V, E)$, a vertex cover is a subset U of vertices such that for every $(u, v) \in E$, either u or v (or both) is in U. The figure below shows a vertex cover for a tree, where the shaded vertices are in U.



Argue that the following greedy algorithm will give a vertex cover of the smallest size. At each step, the algorithm maintains a forest F, initialized with T.

1. Find a leaf v in F.
2. Add the vertex u that is adjacent to v (connected to v through edge (u,v)) to U.
3. Remove all edges and leaf vertices in F that is connected to u.
4. Go back to step 1 until all edges have been removed.

Solution: First we show optimal substructure. Assume an optimal solution U. Consider any vertex v in U. Let the leaf vertices adjacent to v be $L(v)$ and edges incident to v be $E(v)$. Then the size of the smallest cover $S(V,E)$ satisfies $S(V,E) = 1 + S(V\text{-}L(v),E\text{-}E(v))$. To see this, first note that the edges in $E(v)$ have all been covered, so the remaining edges to be covered are in the forest $(V\text{-}L(v),E\text{-}E(v))$. We must use an optimal cover for this forest, otherwise we can replace it with a smaller cover and obtain a set smaller than U that covers F, hence contradicting the optimality of U.

Now we show the greedy choice property. Assume an optimal cover U and that the vertex u selected by the algorithm is not in the optimal cover. The vertex u is connected to a leaf v, hence one of u or v must be in U to cover the edge (u,v). Since u is not in U, v must be in U. We can swap u for v in U and continue to cover all the edges as v covers only the edge (u,v) and that edge is covered by u as well. Hence there exists a cover of the same size as U that contains u.

The optimal substructure property implies that if we are able to select an element from an optimal cover, we can recursively solve the optimal subproblem in order to solve the original problem. The greedy choice property implies that the greedy algorithm will select an element from an optimal cover. Hence, together they imply that the algorithm works correctly.

END OF EXAM